8.4 Tree Traversal Algorithms

A ***traversal*** of a tree T is a systematic way of accessing, or "visiting," all the positions of T. The specific action associated with the "visit" of a position p depends on the application of this traversal, and could involve anything from incrementing a counter to performing some complex computation for p. In this section, we describe several common traversal schemes for trees, implement them in the context of our various tree classes, and discuss several common applications of tree traversals.

8.4.1 Preorder and Postorder Traversals of General Trees

In a ***preorder traversal*** of a tree T, the root of T is visited first and then the subtrees rooted at its children are traversed recursively. If the tree is ordered, then the subtrees are traversed according to the order of the children. The pseudo-code for the preorder traversal of the subtree rooted at a position p is shown in Code Fragment 8.12.

```
Algorithm preorder(T, p):
  perform the "visit" action for position p
  for each child c in T.children(p) do
    preorder(T, c)                {recursively traverse the subtree rooted at c}
```

Code Fragment 8.12: Algorithm preorder for performing the preorder traversal of a subtree rooted at position p of a tree T.

Figure 8.15 portrays the order in which positions of a sample tree are visited during an application of the preorder traversal algorithm.

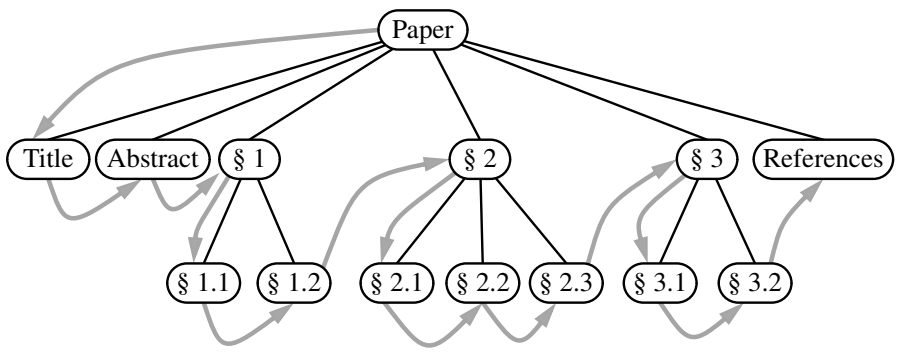


Figure 8.15: Preorder traversal of an ordered tree, where the children of each position are ordered from left to right.

Postorder Traversal

Another important tree traversal algorithm is the ***postorder traversal***. In some sense, this algorithm can be viewed as the opposite of the preorder traversal, because it recursively traverses the subtrees rooted at the children of the root first, and then visits the root (hence, the name "postorder"). Pseudo-code for the postorder traversal is given in Code Fragment 8.13, and an example of a postorder traversal is portrayed in Figure 8.16.

```
Algorithm postorder(T, p):
  for each child c in T.children(p) do
    postorder(T, c)                    {recursively traverse the subtree rooted at c}
  perform the "visit" action for position p
```

Code Fragment 8.13: Algorithm postorder for performing the postorder traversal of a subtree rooted at position p of a tree T.

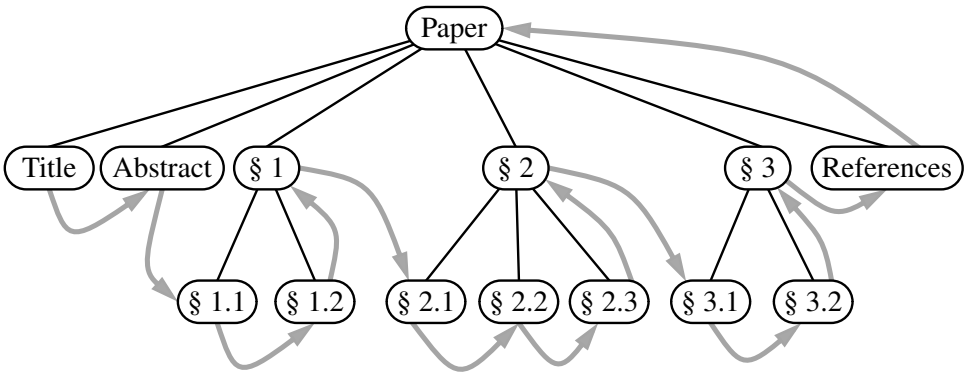


Figure 8.16: Postorder traversal of the ordered tree of Figure 8.15.

Running-Time Analysis

Both preorder and postorder traversal algorithms are efficient ways to access all the positions of a tree. The analysis of either of these traversal algorithms is similar to that of algorithm height2, given in Code Fragment 8.5 of Section 8.1.3. At each position p, the nonrecursive part of the traversal algorithm requires time $O(c_p + 1)$, where c_p is the number of children of p, under the assumption that the "visit" itself takes $O(1)$ time. By Proposition 8.5, the overall running time for the traversal of tree T is $O(n)$, where n is the number of positions in the tree. This running time is asymptotically optimal since the traversal must visit all the n positions of the tree.

8.4.2 Breadth-First Tree Traversal

Although the preorder and postorder traversals are common ways of visiting the positions of a tree, another common approach is to traverse a tree so that we visit all the positions at depth d before we visit the positions at depth $d+1$. Such an algorithm is known as a ***breadth-first traversal***.

A breadth-first traversal is a common approach used in software for playing games. A ***game tree*** represents the possible choices of moves that might be made by a player (or computer) during a game, with the root of the tree being the initial configuration for the game. For example, Figure 8.17 displays a partial game tree for Tic-Tac-Toe.

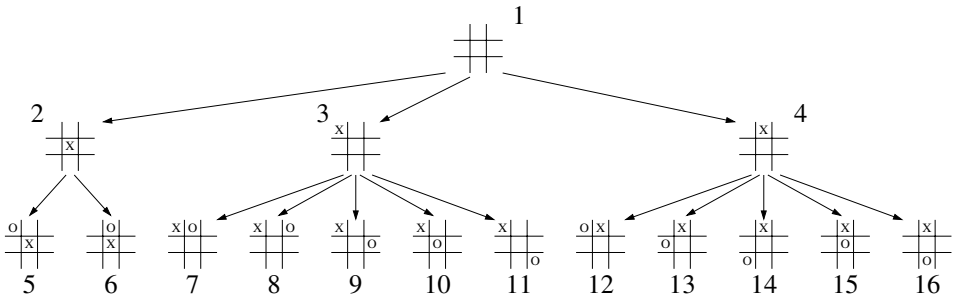


Figure 8.17: Partial game tree for Tic-Tac-Toe, with annotations displaying the order in which positions are visited in a breadth-first traversal.

A breadth-first traversal of such a game tree is often performed because a computer may be unable to explore a complete game tree in a limited amount of time. So the computer will consider all moves, then responses to those moves, going as deep as computational time allows.

Pseudo-code for a breadth-first traversal is given in Code Fragment 8.14. The process is not recursive, since we are not traversing entire subtrees at once. We use a queue to produce a FIFO (i.e., first-in first-out) semantics for the order in which we visit nodes. The overall running time is $O(n)$, due to the n calls to enqueue and n calls to dequeue.

```
Algorithm breadthfirst(T):
  Initialize queue Q to contain T.root( )
  while Q not empty do
    p = Q.dequeue( )                    {p is the oldest entry in the queue}
    perform the "visit" action for position p
    for each child c in T.children(p) do
      Q.enqueue(c)     {add p's children to the end of the queue for later visits}
```

Code Fragment 8.14: Algorithm for performing a breadth-first traversal of a tree.

8.4.3 Inorder Traversal of a Binary Tree

The standard preorder, postorder, and breadth-first traversals that were introduced for general trees, can be directly applied to binary trees. In this section, we introduce another common traversal algorithm specifically for a binary tree.

During an ***inorder traversal***, we visit a position between the recursive traversals of its left and right subtrees. The inorder traversal of a binary tree T can be informally viewed as visiting the nodes of T "from left to right." Indeed, for every position p, the inorder traversal visits p after all the positions in the left subtree of p and before all the positions in the right subtree of p. Pseudo-code for the inorder traversal algorithm is given in Code Fragment 8.15, and an example of an inorder traversal is portrayed in Figure 8.18.

```
Algorithm inorder(p):
  if p has a left child lc then
    inorder(lc)                          {recursively traverse the left subtree of p}
  perform the "visit" action for position p
  if p has a right child rc then
    inorder(rc)                          {recursively traverse the right subtree of p}
```

Code Fragment 8.15: Algorithm inorder for performing an inorder traversal of a subtree rooted at position p of a binary tree.

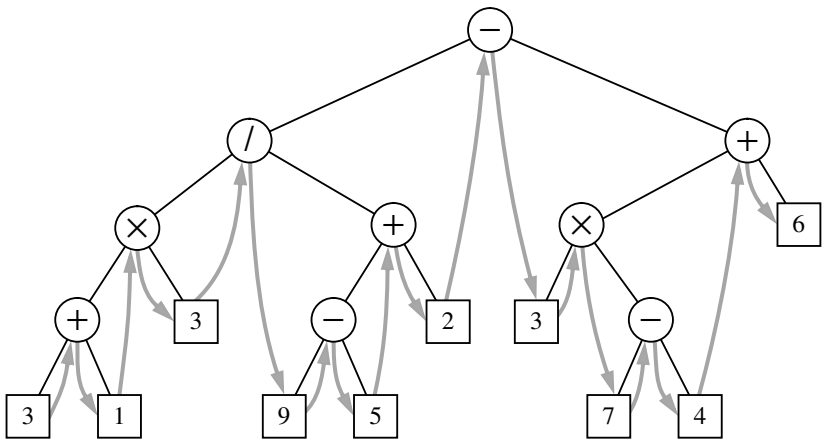


Figure 8.18: Inorder traversal of a binary tree.

The inorder traversal algorithm has several important applications. When using a binary tree to represent an arithmetic expression, as in Figure 8.18, the inorder traversal visits positions in a consistent order with the standard representation of the expression, as in $3+1\times 3/9-5+2\ldots$ (albeit without parentheses).

Binary Search Trees

An important application of the inorder traversal algorithm arises when we store an ordered sequence of elements in a binary tree, defining a structure we call a ***binary search tree***. Let S be a set whose unique elements have an order relation. For example, S could be a set of integers. A binary search tree for S is a binary tree T such that, for each position p of T:

- Position p stores an element of S, denoted as $e(p)$.
- Elements stored in the left subtree of p (if any) are less than $e(p)$.
- Elements stored in the right subtree of p (if any) are greater than $e(p)$.

An example of a binary search tree is shown in Figure 8.19. The above properties assure that an inorder traversal of a binary search tree T visits the elements in nondecreasing order.

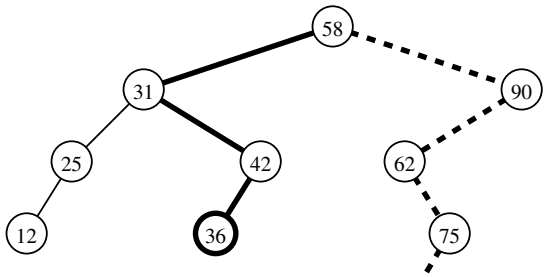


Figure 8.19: A binary search tree storing integers. The solid path is traversed when searching (successfully) for 36. The dashed path is traversed when searching (unsuccessfully) for 70.

We can use a binary search tree T for set S to find whether a given search value v is in S, by traversing a path down the tree T, starting at the root. At each internal position p encountered, we compare our search value v with the element $e(p)$ stored at p. If $v < e(p)$, then the search continues in the left subtree of p. If $v = e(p)$, then the search terminates successfully. If $v > e(p)$, then the search continues in the right subtree of p. Finally, if we reach an empty subtree, the search terminates unsuccessfully. In other words, a binary search tree can be viewed as a binary decision tree (recall Example 8.6), where the question asked at each internal node is whether the element at that node is less than, equal to, or larger than the element being searched for. We illustrate several examples of the search operation in Figure 8.19.

Note that the running time of searching in a binary search tree T is proportional to the height of T. Recall from Proposition 8.8 that the height of a binary tree with n nodes can be as small as $\log(n+1)-1$ or as large as $n-1$. Thus, binary search trees are most efficient when they have small height. Chapter 11 is devoted to the study of search trees.

8.4.4 Implementing Tree Traversals in Python

When first defining the tree ADT in Section 8.1.2, we stated that tree T should include support for the following methods:

T.positions( ): Generate an iteration of all *positions* of tree T.

iter(T): Generate an iteration of all *elements* stored within tree T.

At that time, we did not make any assumption about the order in which these iterations report their results. In this section, we demonstrate how any of the tree traversal algorithms we have introduced could be used to produce these iterations.

To begin, we note that it is easy to produce an iteration of all elements of a tree, if we rely on a presumed iteration of all positions. Therefore, support for the iter(T) syntax can be formally provided by a concrete implementation of the special method __iter__ within the abstract base class Tree. We rely on Python's generator syntax as the mechanism for producing iterations. (See Section 1.8.) Our implementation of Tree.__iter__ is given in Code Fragment 8.16.

```
75  def __iter__(self):
76    """Generate an iteration of the tree's elements."""
77    for p in self.positions( ):        # use same order as positions()
78      yield p.element( )               # but yield each element
```

Code Fragment 8.16: Iterating all elements of a Tree instance, based upon an iteration of the positions of the tree. This code should be included in the body of the Tree class.

To implement the positions method, we have a choice of tree traversal algorithms. Given that there are advantages to each of those traversal orders, we will provide independent implementations of each strategy that can be called directly by a user of our class. We can then trivially adapt one of those as a default order for the positions method of the tree ADT.

Preorder Traversal

We begin by considering the ***preorder traversal*** algorithm. We will support a public method with calling signature T.preorder( ) for tree T, which generates a preorder iteration of all positions within the tree. However, the recursive algorithm for generating a preorder traversal, as originally described in Code Fragment 8.12, must be parameterized by a specific position within the tree that serves as the root of a subtree to traverse. A standard solution for such a circumstance is to define a nonpublic utility method with the desired recursive parameterization, and then to have the public method preorder invoke the nonpublic method upon the root of the tree. Our implementation of such a design is given in Code Fragment 8.17.

```
79   def preorder(self):
80     """Generate a preorder iteration of positions in the tree."""
81     if not self.is_empty( ):
82       for p in self._subtree_preorder(self.root( )):       # start recursion
83         yield p
84
85   def _subtree_preorder(self, p):
86     """Generate a preorder iteration of positions in subtree rooted at p."""
87     yield p                                    # visit p before its subtrees
88     for c in self.children(p):                 # for each child c
89       for other in self._subtree_preorder(c):  # do preorder of c's subtree
90         yield other                            # yielding each to our caller
```

Code Fragment 8.17: Support for performing a preorder traversal of a tree. This code should be included in the body of the Tree class.

Formally, both preorder and the utility _subtree_preorder are generators. Rather than perform a "visit" action from within this code, we yield each position to the caller and let the caller decide what action to perform at that position.

The _subtree_preorder method is the recursive one. However, because we are relying on generators rather than traditional functions, the recursion has a slightly different form. In order to yield all positions within the subtree of child c, we loop over the positions yielded by the recursive call self._subtree_preorder(c), and re-yield each position in the outer context. Note that if p is a leaf, the for loop over self.children(p) is trivial (this is the base case for our recursion).

We rely on a similar technique in the public preorder method to re-yield all positions that are generated by the recursive process starting at the root of the tree; if the tree is empty, nothing is yielded. At this point, we have provided full support for the preorder generator. A user of the class can therefore write code such as

```
for p in T.preorder( ):
  # "visit" position p
```

The official tree ADT requires that all trees support a positions method as well. To use a preorder traversal as the default order of iteration, we include the definition shown in Code Fragment 8.18 within our Tree class. Rather than loop over the results returned by the preorder call, we return the entire iteration as an object.

```
91   def positions(self):
92     """Generate an iteration of the tree's positions."""
93     return self.preorder( )              # return entire preorder iteration
```

Code Fragment 8.18: An implementation of the positions method for the Tree class that relies on a preorder traversal to generate the results.

Postorder Traversal

We can implement a postorder traversal using very similar technique as with a preorder traversal. The only difference is that within the recursive utility for a postorder we wait to yield position p until *after* we have recursively yield the positions in its subtrees. An implementation is given in Code Fragment 8.19.

```
94   def postorder(self):
95     """Generate a postorder iteration of positions in the tree."""
96     if not self.is_empty( ):
97       for p in self._subtree_postorder(self.root( )):       # start recursion
98         yield p
99
100  def _subtree_postorder(self, p):
101    """Generate a postorder iteration of positions in subtree rooted at p."""
102    for c in self.children(p):                    # for each child c
103      for other in self._subtree_postorder(c):    # do postorder of c's subtree
104        yield other                               # yielding each to our caller
105    yield p                                       # visit p after its subtrees
```

Code Fragment 8.19: Support for performing a postorder traversal of a tree. This code should be included in the body of the Tree class.

Breadth-First Traversal

In Code Fragment 8.20, we provide an implementation of the breadth-first traversal algorithm in the context of our Tree class. Recall that the breadth-first traversal algorithm is not recursive; it relies on a queue of positions to manage the traversal process. Our implementation uses the LinkedQueue class from Section 7.1.2, although any implementation of the queue ADT would suffice.

Inorder Traversal for Binary Trees

The preorder, postorder, and breadth-first traversal algorithms are applicable to all trees, and so we include their implementations within the Tree abstract base class. Those methods are inherited by the abstract BinaryTree class, the concrete LinkedBinaryTree class, and any other dependent tree classes we might develop.

The inorder traversal algorithm, because it explicitly relies on the notion of a left and right child of a node, only applies to binary trees. We therefore include its definition within the body of the BinaryTree class. We use a similar technique to implement an inorder traversal (Code Fragment 8.21) as we did with preorder and postorder traversals.

```
106   def breadthfirst(self):
107     """Generate a breadth-first iteration of the positions of the tree."""
108     if not self.is_empty( ):
109       fringe = LinkedQueue( )                # known positions not yet yielded
110       fringe.enqueue(self.root( ))           # starting with the root
111       while not fringe.is_empty( ):
112         p = fringe.dequeue( )                # remove from front of the queue
113         yield p                              # report this position
114         for c in self.children(p):
115           fringe.enqueue(c)                  # add children to back of queue
```

Code Fragment 8.20: An implementation of a breadth-first traversal of a tree. This code should be included in the body of the Tree class.

```
37   def inorder(self):
38     """Generate an inorder iteration of positions in the tree."""
39     if not self.is_empty( ):
40       for p in self._subtree_inorder(self.root( )):
41         yield p
42
43   def _subtree_inorder(self, p):
44     """Generate an inorder iteration of positions in subtree rooted at p."""
45     if self.left(p) is not None:          # if left child exists, traverse its subtree
46       for other in self._subtree_inorder(self.left(p)):
47         yield other
48     yield p                               # visit p between its subtrees
49     if self.right(p) is not None:         # if right child exists, traverse its subtree
50       for other in self._subtree_inorder(self.right(p)):
51         yield other
```

Code Fragment 8.21: Support for performing an inorder traversal of a binary tree. This code should be included in the BinaryTree class (given in Code Fragment 8.7).

For many applications of binary trees, an inorder traversal provides a natural iteration. We could make it the default for the BinaryTree class by overriding the positions method that was inherited from the Tree class (see Code Fragment 8.22).

```
52   # override inherited version to make inorder the default
53   def positions(self):
54     """Generate an iteration of the tree's positions."""
55     return self.inorder( )                # make inorder the default
```

Code Fragment 8.22: Defining the BinaryTree.position method so that positions are reported using inorder traversal.

8.4.5 Applications of Tree Traversals

In this section, we demonstrate several representative applications of tree traversals, including some customizations of the standard traversal algorithms.

Table of Contents

When using a tree to represent the hierarchical structure of a document, a preorder traversal of the tree can naturally be used to produce a table of contents for the document. For example, the table of contents associated with the tree from Figure 8.15 is displayed in Figure 8.20. Part (a) of that figure gives a simple presentation with one element per line; part (b) shows a more attractive presentation produced by indenting each element based on its depth within the tree. A similar presentation could be used to display the contents of a computer's file system, based on its tree representation (as in Figure 8.3).

```
Paper                Paper
Title                  Title
Abstract               Abstract
§1                     §1
§1.1                     §1.1
§1.2                     §1.2
§2                     §2
§2.1                     §2.1
...                      ...

(a)                  (b)
```

Figure 8.20: Table of contents for a document represented by the tree in Figure 8.15: (a) without indentation; (b) with indentation based on depth within the tree.

The unindented version of the table of contents, given a tree T, can be produced with the following code:

```
for p in T.preorder( ):
  print(p.element( ))
```

To produce the presentation of Figure 8.20(b), we indent each element with a number of spaces equal to twice the element's depth in the tree (hence, the root element was unindented). Although we could replace the body of the above loop with the statement `print(2*T.depth(p)*' ' + str(p.element( )))`, such an approach is unnecessarily inefficient. Although the work to produce the preorder traversal runs in $O(n)$ time, based on the analysis of Section 8.4.1, the calls to `depth` incur a hidden cost. Making a call to `depth` from every position of the tree results in $O(n^2)$ worst-case time, as noted when analyzing the algorithm `height1` in Section 8.1.3.

A preferred approach to producing an indented table of contents is to redesign a top-down recursion that includes the current depth as an additional parameter. Such an implementation is provided in Code Fragment 8.23. This implementation runs in worst-case $O(n)$ time (except, technically, the time it takes to print strings of increasing lengths).

```
1  def preorder_indent(T, p, d):
2    """Print preorder representation of subtree of T rooted at p at depth d."""
3    print(2*d*' ' + str(p.element( )))          # use depth for indentation
4    for c in T.children(p):
5      preorder_indent(T, c, d+1)                # child depth is d+1
```

Code Fragment 8.23: Efficient recursion for printing indented version of a preorder traversal. On a complete tree T, the recursion should be started with form preorder_indent(T, T.root( ), 0).

In the example of Figure 8.20, we were fortunate in that the numbering was embedded within the elements of the tree. More generally, we might be interested in using a preorder traversal to display the structure of a tree, with indentation and also explicit numbering that was not present in the tree. For example, we might display the tree from Figure 8.2 beginning as:

```
Electronics R'Us
  1 R&D
  2 Sales
    2.1 Domestic
    2.2 International
      2.2.1 Canada
      2.2.2 S. America
```

This is more challenging, because the numbers used as labels are implicit in the structure of the tree. A label depends on the index of each position, relative to its siblings, along the path from the root to the current position. To accomplish the task, we add a representation of that path as an additional parameter to the recursive signature. Specifically, we use a list of zero-indexed numbers, one for each position along the downward path, other than the root. (We convert those numbers to one-indexed form when printing.)

At the implementation level, we wish to avoid the inefficiency of duplicating such lists when sending a new parameter from one level of the recursion to the next. A standard solution is to share the same list instance throughout the recursion. At one level of the recursion, a new entry is temporarily added to the end of the list before making further recursive calls. In order to "leave no trace," that same block of code must remove the extraneous entry from the list before completing its task. An implementation based on this approach is given in Code Fragment 8.24.

```
1  def preorder_label(T, p, d, path):
2    """Print labeled representation of subtree of T rooted at p at depth d."""
3    label = '.'.join(str(j+1) for j in path)    # displayed labels are one-indexed
4    print(2*d*'  ' + label, p.element())
5    path.append(0)                              # path entries are zero-indexed
6    for c in T.children(p):
7      preorder_label(T, c, d+1, path)           # child depth is d+1
8      path[-1] += 1
9    path.pop()
```

Code Fragment 8.24: Efficient recursion for printing an indented and *labeled* presentation of a preorder traversal.

Parenthetic Representations of a Tree

It is not possible to reconstruct a general tree, given only the preorder sequence of elements, as in Figure 8.20(a). Some additional context is necessary for the structure of the tree to be well defined. The use of indentation or numbered labels provides such context, with a very human-friendly presentation. However, there are more concise string representations of trees that are computer-friendly.

In this section, we explore one such representation. The ***parenthetic string representation*** $P(T)$ of tree T is recursively defined as follows. If T consists of a single position p, then

$$P(T) = \mathsf{str}(p.\mathsf{element}()).$$

Otherwise, it is defined recursively as,

$$P(T) = \mathsf{str}(p.\mathsf{element}()) + \texttt{'('} + P(T_1) + \texttt{', '} + \cdots + \texttt{', '} + P(T_k) + \texttt{')'}$$

where p is the root of T and $T_1, T_2, \ldots, T_k$ are the subtrees rooted at the children of p, which are given in order if T is an ordered tree. We are using "+" here to denote string concatenation. As an example, the parenthetic representation of the tree of Figure 8.2 would appear as follows (line breaks are cosmetic):

```
Electronics R'Us (R&D, Sales (Domestic, International (Canada,
S. America, Overseas (Africa, Europe, Asia, Australia))),
Purchasing, Manufacturing (TV, CD, Tuner))
```

Although the parenthetic representation is essentially a preorder traversal, we cannot easily produce the additional punctuation using the formal implementation of preorder, as given in Code Fragment 8.17. The opening parenthesis must be produced just before the loop over a position's children and the closing parenthesis must be produced just after that loop. Furthermore, the separating commas must be produced. The Python function parenthesize, shown in Code Fragment 8.25, is a custom traversal that prints such a parenthetic string representation of a tree T.

```
def parenthesize(T, p):
  """Print parenthesized representation of subtree of T rooted at p."""
  print(p.element( ), end='')          # use of end avoids trailing newline
  if not T.is_leaf(p):
    first_time = True
    for c in T.children(p):
      sep = ' (' if first_time else ', '          # determine proper separator
      print(sep, end='')
      first_time = False                # any future passes will not be the first
      parenthesize(T, c)                # recur on child
    print(')', end='')                  # include closing parenthesis
```

Code Fragment 8.25: Function that prints parenthetic string representation of a tree.

Computing Disk Space

In Example 8.1, we considered the use of a tree as a model for a file-system structure, with internal positions representing directories and leaves representing files. In fact, when introducing the use of recursion back in Chapter 4, we specifically examined the topic of file systems (see Section 4.1.4). Although we did not explicitly model it as a tree at that time, we gave an implementation of an algorithm for computing the disk usage (Code Fragment 4.5).

The recursive computation of disk space is emblematic of a *postorder* traversal, as we cannot effectively compute the total space used by a directory until *after* we know the space that is used by its children directories. Unfortunately, the formal implementation of postorder, as given in Code Fragment 8.19 does not suffice for this purpose. As it visits the position of a directory, there is no easy way to discern which of the previous positions represent children of that directory, nor how much recursive disk space was allocated.

We would like to have a mechanism for children to return information to the parent as part of the traversal process. A custom solution to the disk space problem, with each level of recursion providing a return value to the (parent) caller, is provided in Code Fragment 8.26.

```
def disk_space(T, p):
  """Return total disk space for subtree of T rooted at p."""
  subtotal = p.element( ).space( )          # space used at position p
  for c in T.children(p):
    subtotal += disk_space(T, c)            # add child's space to subtotal
  return subtotal
```

Code Fragment 8.26: Recursive computation of disk space for a tree. We assume that a space( ) method of each tree element reports the local space used at that position.

8.4.6 Euler Tours and the Template Method Pattern ⋆

The various applications described in Section 8.4.5 demonstrate the great power of recursive tree traversals. Unfortunately, they also show that the specific implementations of the preorder and postorder methods of our Tree class, or the inorder method of the BinaryTree class, are not general enough to capture the range of computations we desire. In some cases, we need more of a blending of the approaches, with initial work performed before recurring on subtrees, additional work performed after those recursions, and in the case of a binary tree, work performed between the two possible recursions. Furthermore, in some contexts it was important to know the depth of a position, or the complete path from the root to that position, or to return information from one level of the recursion to another. For each of the previous applications, we were able to develop a custom implementation to properly adapt the recursive ideas, but the great principles of object-oriented programming introduced in Section 2.1.1 include ***adaptability*** and ***reusability***.

In this section, we develop a more general framework for implementing tree traversals based on a concept known as an ***Euler tour traversal***. The Euler tour traversal of a general tree T can be informally defined as a "walk" around T, where we start by going from the root toward its leftmost child, viewing the edges of T as being "walls" that we always keep to our left. (See Figure 8.21.)

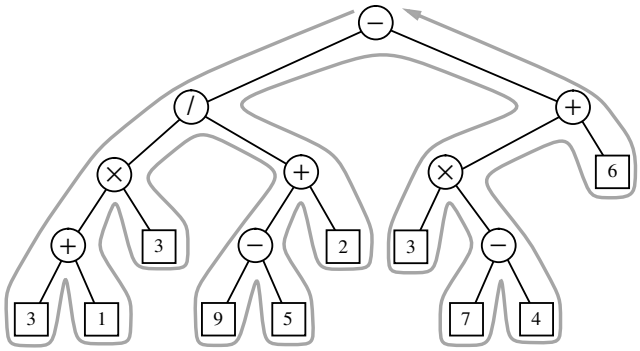


Figure 8.21: Euler tour traversal of a tree.

The complexity of the walk is $O(n)$, because it progresses exactly two times along each of the $n-1$ edges of the tree—once going downward along the edge, and later going upward along the edge. To unify the concept of preorder and postorder traversals, we can think of there being two notable "visits" to each position p:

- A "pre visit" occurs when first reaching the position, that is, when the walk passes immediately left of the node in our visualization.
- A "post visit" occurs when the walk later proceeds upward from that position, that is, when the walk passes to the right of the node in our visualization.

The process of an Euler tour can easily be viewed recursively. In between the "pre visit" and "post visit" of a given position will be a recursive tour of each of its subtrees. Looking at Figure 8.21 as an example, there is a contiguous portion of the entire tour that is itself an Euler tour of the subtree of the node with element "/". That tour contains two contiguous subtours, one traversing that position's left subtree and another traversing the right subtree. The pseudo-code for an Euler tour traversal of a subtree rooted at a position p is shown in Code Fragment 8.27.

```
Algorithm eulertour(T, p):
  perform the "pre visit" action for position p
  for each child c in T.children(p) do
    eulertour(T, c)                    {recursively tour the subtree rooted at c}
  perform the "post visit" action for position p
```

Code Fragment 8.27: Algorithm eulertour for performing an Euler tour traversal of a subtree rooted at position p of a tree.

The Template Method Pattern

To provide a framework that is reusable and adaptable, we rely on an interesting object-oriented software design pattern, the ***template method pattern***. The template method pattern describes a generic computation mechanism that can be specialized for a particular application by redefining certain steps. To allow customization, the primary algorithm calls auxiliary functions known as ***hooks*** at designated steps of the process.

In the context of an Euler tour traversal, we define two separate hooks, a previsit hook that is called before the subtrees are traversed, and a postvisit hook that is called after the completion of the subtree traversals. Our implementation will take the form of an EulerTour class that manages the process, and defines trivial definitions for the hooks that do nothing. The traversal can be customized by defining a subclass of EulerTour and overriding one or both hooks to provide specialized behavior.

Python Implementation

Our implementation of an EulerTour class is provided in Code Fragment 8.28. The primary recursive process is defined in the nonpublic _tour method. A tour instance is created by sending a reference to a specific tree to the constructor, and then by calling the public execute method, which beings the tour and returns a final result of the computation.

```
1  class EulerTour:
2    """Abstract base class for performing Euler tour of a tree.
3
4    _hook_previsit and _hook_postvisit may be overridden by subclasses.
5    """
6    def __init__(self, tree):
7      """Prepare an Euler tour template for given tree."""
8      self._tree = tree
9
10   def tree(self):
11     """Return reference to the tree being traversed."""
12     return self._tree
13
14   def execute(self):
15     """Perform the tour and return any result from post visit of root."""
16     if len(self._tree) > 0:
17       return self._tour(self._tree.root( ), 0, [ ])          # start the recursion
18
19   def _tour(self, p, d, path):
20     """Perform tour of subtree rooted at Position p.
21
22     p          Position of current node being visited
23     d          depth of p in the tree
24     path       list of indices of children on path from root to p
25     """
26     self._hook_previsit(p, d, path)                          # "pre visit" p
27     results = [ ]
28     path.append(0)             # add new index to end of path before recursion
29     for c in self._tree.children(p):
30       results.append(self._tour(c, d+1, path))      # recur on child's subtree
31       path[-1] += 1             # increment index
32     path.pop( )                 # remove extraneous index from end of path
33     answer = self._hook_postvisit(p, d, path, results)        # "post visit" p
34     return answer
35
36   def _hook_previsit(self, p, d, path):                      # can be overridden
37     pass
38
39   def _hook_postvisit(self, p, d, path, results):            # can be overridden
40     pass
```

Code Fragment 8.28: An EulerTour base class providing a framework for performing Euler tour traversals of a tree.

Based on our experience of customizing traversals for sample applications Section 8.4.5, we build support into the primary EulerTour for maintaining the recursive depth and the representation of the recursive path through a tree, using the approach that we introduced in Code Fragment 8.24. We also provide a mechanism for one recursive level to return a value to another when post-processing. Formally, our framework relies on the following two hooks that can be specialized:

- method _hook_previsit(p, d, path)
  This function is called once for each position, immediately before its subtrees (if any) are traversed. Parameter p is a position in the tree, d is the depth of that position, and path is a list of indices, using the convention described in the discussion of Code Fragment 8.24. No return value is expected from this function.
- method _hook_postvisit(p, d, path, results)
  This function is called once for each position, immediately after its subtrees (if any) are traversed. The first three parameters use the same convention as did _hook_previsit. The final parameter is a list of objects that were provided as return values from the post visits of the respective subtrees of p. Any value returned by this call will be available to the parent of p during its postvisit.

For more complex tasks, subclasses of EulerTour may also choose to initialize and maintain additional state in the form of instance variables that can be accessed within the bodies of the hooks.

Using the Euler Tour Framework

To demonstrate the flexibility of our Euler tour framework, we revisit the sample applications from Section 8.4.5. As a simple example, an indented preorder traversal, akin to that originally produced by Code Fragment 8.23, can be generated with the simple subclass given in Code Fragment 8.29.

```
1 class PreorderPrintIndentedTour(EulerTour):
2   def _hook_previsit(self, p, d, path):
3     print(2*d*' ' + str(p.element( )))
```

Code Fragment 8.29: A subclass of EulerTour that produces an indented preorder list of a tree's elements.

Such a tour would be started by creating an instance of the subclass for a given tree T, and invoking its execute method. This could be expressed as follows:

```
tour = PreorderPrintIndentedTour(T)
tour.execute( )
```

A labeled version of an indented, preorder presentation, akin to Code Fragment 8.24, could be generated by the new subclass of EulerTour shown in Code Fragment 8.30.

```
1  class PreorderPrintIndentedLabeledTour(EulerTour):
2    def _hook_previsit(self, p, d, path):
3      label = '.'.join(str(j+1) for j in path)    # labels are one-indexed
4      print(2*d*' ' + label, p.element( ))
```

Code Fragment 8.30: A subclass of EulerTour that produces a labeled and indented, preorder list of a tree's elements.

To produce the parenthetic string representation, originally achieved with Code Fragment 8.25, we define a subclass that overrides both the previsit and postvisit hooks. Our new implementation is given in Code Fragment 8.31.

```
1  class ParenthesizeTour(EulerTour):
2    def _hook_previsit(self, p, d, path):
3      if path and path[-1] > 0:                   # p follows a sibling
4        print(', ', end='')                       # so preface with comma
5      print(p.element( ), end='')                 # then print element
6      if not self.tree( ).is_leaf(p):             # if p has children
7        print(' (', end='')                       # print opening parenthesis
8
9    def _hook_postvisit(self, p, d, path, results):
10     if not self.tree( ).is_leaf(p):             # if p has children
11       print(')', end='')                        # print closing parenthesis
```

Code Fragment 8.31: A subclass of EulerTour that prints a parenthetic string representation of a tree.

Notice that in this implementation, we need to invoke a method on the tree instance that is being traversed from within the hooks. The public tree( ) method of the EulerTour class serves as an accessor for that tree.

Finally, the task of computing disk space, as originally implemented in Code Fragment 8.26, can be performed quite easily with the EulerTour subclass shown in Code Fragment 8.32. The postvisit result of the root will be returned by the call to execute( ).

```
1  class DiskSpaceTour(EulerTour):
2    def _hook_postvisit(self, p, d, path, results):
3      # we simply add space associated with p to that of its subtrees
4      return p.element( ).space( ) + sum(results)
```

Code Fragment 8.32: A subclass of EulerTour that computes disk space for a tree.

The Euler Tour Traversal of a Binary Tree

In Section 8.4.6, we introduced the concept of an Euler tour traversal of a general graph, using the template method pattern in designing the EulerTour class. That class provided methods _hook_previsit and _hook_postvisit that could be overridden to customize a tour. In Code Fragment 8.33 we provide a BinaryEulerTour specialization that includes an additional _hook_invisit that is called once for each position—after its left subtree is traversed, but before its right subtree is traversed.

Our implementation of BinaryEulerTour replaces the original _tour utility to specialize to the case in which a node has at most two children. If a node has only one child, a tour differentiates between whether that is a left child or a right child, with the "in visit" taking place after the visit of a sole left child, but before the visit of a sole right child. In the case of a leaf, the three hooks are called in succession.

```
1  class BinaryEulerTour(EulerTour):
2    """Abstract base class for performing Euler tour of a binary tree.
3
4    This version includes an additional _hook_invisit that is called after the tour
5    of the left subtree (if any), yet before the tour of the right subtree (if any).
6
7    Note: Right child is always assigned index 1 in path, even if no left sibling.
8    """
9    def _tour(self, p, d, path):
10     results = [None, None]          # will update with results of recursions
11     self._hook_previsit(p, d, path)                    # "pre visit" for p
12     if self._tree.left(p) is not None:                 # consider left child
13       path.append(0)
14       results[0] = self._tour(self._tree.left(p), d+1, path)
15       path.pop( )
16     self._hook_invisit(p, d, path)                     # "in visit" for p
17     if self._tree.right(p) is not None:                # consider right child
18       path.append(1)
19       results[1] = self._tour(self._tree.right(p), d+1, path)
20       path.pop( )
21     answer = self._hook_postvisit(p, d, path, results)       # "post visit" p
22     return answer
23
24   def _hook_invisit(self, p, d, path): pass            # can be overridden
```

Code Fragment 8.33: A BinaryEulerTour base class providing a specialized tour for binary trees. The original EulerTour base class was given in Code Fragment 8.28.

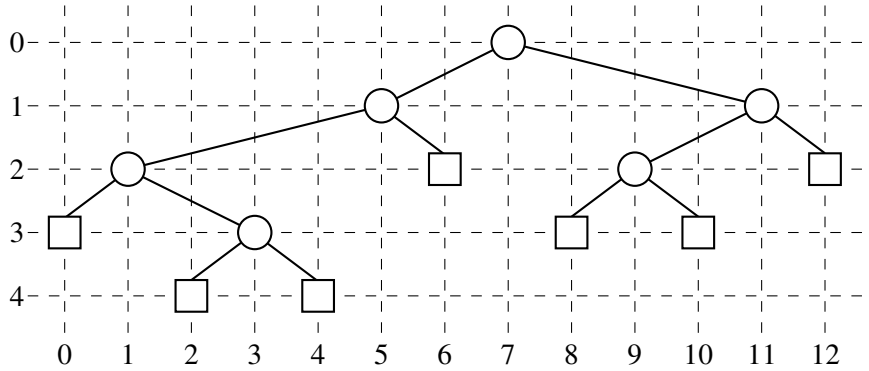


Figure 8.22: An inorder drawing of a binary tree.

To demonstrate use of the BinaryEulerTour framework, we develop a subclass that computes a graphical layout of a binary tree, as shown in Figure 8.22. The geometry is determined by an algorithm that assigns x- and y-coordinates to each position p of a binary tree T using the following two rules:

- $x(p)$ is the number of positions visited before p in an inorder traversal of T.
- $y(p)$ is the depth of p in T.

In this application, we take the convention common in computer graphics that x-coordinates increase left to right and y-coordinates increase top to bottom. So the origin is in the upper left corner of the computer screen.

Code Fragment 8.34 provides an implementation of a BinaryLayout subclass that implements the above algorithm for assigning (x,y) coordinates to the element stored at each position of a binary tree. We adapt the BinaryEulerTour framework by introducing additional state in the form of a _count instance variable that represents the number of "in visits" that we have performed. The x-coordinate for each position is set according to that counter.

```
1  class BinaryLayout(BinaryEulerTour):
2    """Class for computing (x,y) coordinates for each node of a binary tree."""
3    def __init__(self, tree):
4      super().__init__(tree)                # must call the parent constructor
5      self._count = 0                       # initialize count of processed nodes
6
7    def _hook_invisit(self, p, d, path):
8      p.element().setX(self._count)         # x-coordinate serialized by count
9      p.element().setY(d)                   # y-coordinate is depth
10     self._count += 1                      # advance count of processed nodes
```

Code Fragment 8.34: A BinaryLayout class that computes coordinates at which to draw positions of a binary tree. We assume that the element type for the original tree supports setX and setY methods.